

Nazwa wzorca: Iterator

Opis problemu: Iterator to behawioralny wzorec projektowy, pozwalający sekwencyjnie przechodzić od elementu do elementu jakiegoś zbioru bez konieczności poruszania się po nim zależnie od struktury danych jaką jest.

Przykład: jeśli zbiorem wartości jest np macierz to iterator pozwala na przeglądanie wartości nie tylko po kolei, ale też z góry na dół, na skos, od prawej do lewej (zależnie od zaprojektowanego iteratora).

Rozwiązania:

1. Zadeklaruj interfejs iteratora. Wymagane jest zdefiniowanie co najmniej dwóch metod: sprawdzanie czy obecnie przeglądany element ma następcę oraz pobierającej i zwracającej ten element.
2. Zadeklaruj interfejs kolekcji i opisz metodę pobierającą iteratora. Typ zwracany przez metodę powinien być zgodny z interfejsem iteratora.
3. Zaimplementuj klasę konkretnego iteratora. Obiekt tej klasy musi przechowywać wskaźnik do struktury danych oraz licznik bieżącego stanu iteracji.
4. Zaimplementuj metodę sprawdzania czy istnieje następca. Logika funkcji musi opierać się na weryfikacji, czy wartość wskaźnika stanu jest mniejsza od całkowitego rozmiaru iterowanej struktury.
5. Zaimplementuj metodę zwracającą następcę w klasie konkretnego iteratora. Funkcja musi zwracać element znajdujący się pod adresem wskazywanym przez bieżący wskaźnik stanu, a następnie dokonać jego zwiększenia.

Konsekwencje:

Plusy:

- Można przeglądać tę samą kolekcję równolegle wieloma iteratorami, gdyż każdy z nich przechowuje informacje o swoim stanie.
- Zasada otwarte/zamknięte. Można zaimplementować nowe typy kolekcji i iteratorów oraz przekazywać je do istniejącego kodu bez psucia czegokolwiek.

Minusy:

- Zastosowanie tego wzorca będzie przesadą jeśli twoja aplikacja korzysta wyłącznie z prostych kolekcji.
- Używanie iteratora może być mniej efektywne niż bezpośrednie przejście po elementach jakiejś wyspecjalizowanej kolekcji.